StateVerdict

Controlled Fault Exploration & Evidence-Based Business Correctness Testing

FINAL REVIEWED ENGINEERING DESIGN • VERSION 1.0 • 15 OCTOBER 2026

Status: APPROVED DESIGN / NOT YET IMPLEMENTED. Owner: Solo engineering build. Review gate: technical spike first.

Primary promise: Generate bounded, meaningful failure schedules for business workflows; verify important outcomes against authoritative evidence; locally minimize and save reproducible violations.

Out of scope: claims of mathematically complete verification, guaranteed deterministic replay of distributed executions, and detection of every bug.

Document control

Item	Decision
Source of truth	This document; implementation changes require an Architecture Decision Record (ADR).
Next action	One-week dispatcher-and-idempotency technical spike.
Resume point	Update DEVELOPMENT_CHECKPOINT.md after every session.
Release policy	No feature is complete until its acceptance tests run and evidence is recorded.

Contents / reading guide

1 Product problem and audience • 2 Scope and feature register • 3 Architecture • 4 Workflow and scheduler • 5 Rules and verdicts • 6 Exploration and shrinking • 7 Persistence and security • 8 Interfaces and repository • 9 Testing/evaluation • 10 Roadmap and spike • 11 Risks/ADRs • 12 Handoff and recruiter demonstration

1. Product purpose and measurable value

Business problem. Independent services can each behave correctly at the API boundary yet produce an invalid combined business outcome: for example, an order is confirmed after payment was declined or a duplicated delivery causes two payment captures.

Target user. Backend engineers, software quality engineers, and platform engineers responsible for event-driven applications. First supported environment is a local, non-production Spring Boot commerce demonstration.

User journey. Developer defines a normal checkout and rules → StateVerdict generates a valid fault schedule → a controlled dispatcher applies interventions → the engine gathers committed-state evidence → the evaluator issues a verdict → failures are reduced and retained as regression scenarios → CI verifies them on future changes.

Primary value proposition. Make fault discovery, rule evaluation, uncertainty, and repeatable failure evidence usable without requiring custom application-specific test harnesses for every fault permutation. Conventional JUnit/Testcontainers remains the comparison baseline, not a straw man.

2. Locked scope and feature register

Priority	Feature	Done when
P0	Two-service order/payment demo	Normal order causes exactly one capture; invalid variant creates duplicate capture on repeated delivery.
P0	Controlled dispatch scheduler	Can hold/release/duplicate a consumed test message; each intervention recorded with run ID.
P0	Evidence adapter and pure evaluator	Reads committed capture state; same frozen evidence + rule produces same verdict.
P0	CLI + machine-readable reports	Run from terminal; JSON report and meaningful exit code.
P1	Kafka + third inventory service	Models asynchronous checkout and delayed reservation release with tested temporal rules.
P1	Constrained random schedule generator	Produces valid schedules within seed, budget and model constraints.
P1	Locally minimizing shrinker	Removes/simplifies faults, preserves same violation under configured repeated trials.
P1	Evidence and schedule replay	Separately supports deterministic rule reevaluation and probabilistic reexecution.
P1	CI quality gate and mutation study	Critical regressions block pipeline; measured baseline comparison.
P2	Second independent target integration	Existing third-party app used without changing its core business semantics.
P2	Minimal report UI	Read-only run list and detailed evidence page; no large portal.

Explicit non-goals

- Production financial transfers, real payment credentials, or access to live customer data.
- Universal Kafka ordering control or deterministic control of internal thread races.
- Arbitrary user-supplied code execution or unrestricted external network scanning.
- Kubernetes control plane, AI diagnosis, hosted multitenant SaaS, sophisticated frontend, and paid cloud hosting for MVP.

3. Architecture and system boundaries

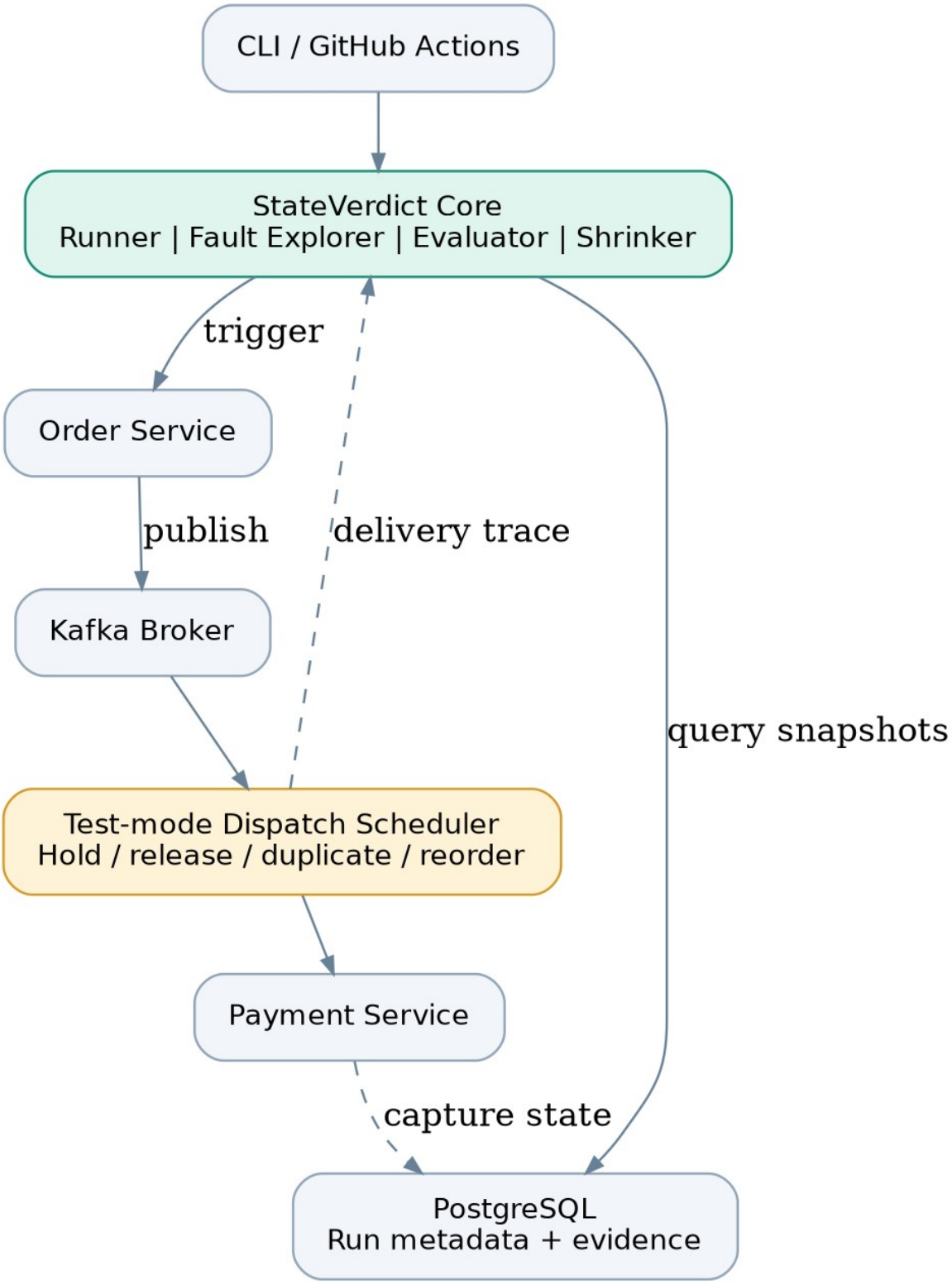


Figure 1. Logical components; scheduled delivery occurs after Kafka consumption and before business handler invocation.

StateVerdict Core is initially a modular monolith with modules for scenario validation, execution, controlled scheduling, evidence acquisition, pure evaluation, fault exploration, shrinking, reporting, and persistence. The order/payment/inventory services are the system under test (SUT), not part of the product control plane.

The test-mode dispatch adapter intercepts Kafka records before domain processing; it records event identity and correlation, then queues logical deliveries for controlled release. It must not claim to control broker internals, real network packet ordering, or service-local races. Production-mode application code should bypass scheduler behavior.

Trust boundaries

- Tests run against allowlisted local or staging endpoints only; reject production-looking endpoints by default and require explicit override in later iterations.
- State snapshots come from read-only, documented state APIs or isolated test databases; trace spans alone do not establish committed state.
- All evidence stores source, observation time, business ID, rule version and hash; secrets and personal data are excluded.

4. End-to-end flow, state machines, and dispatcher contract

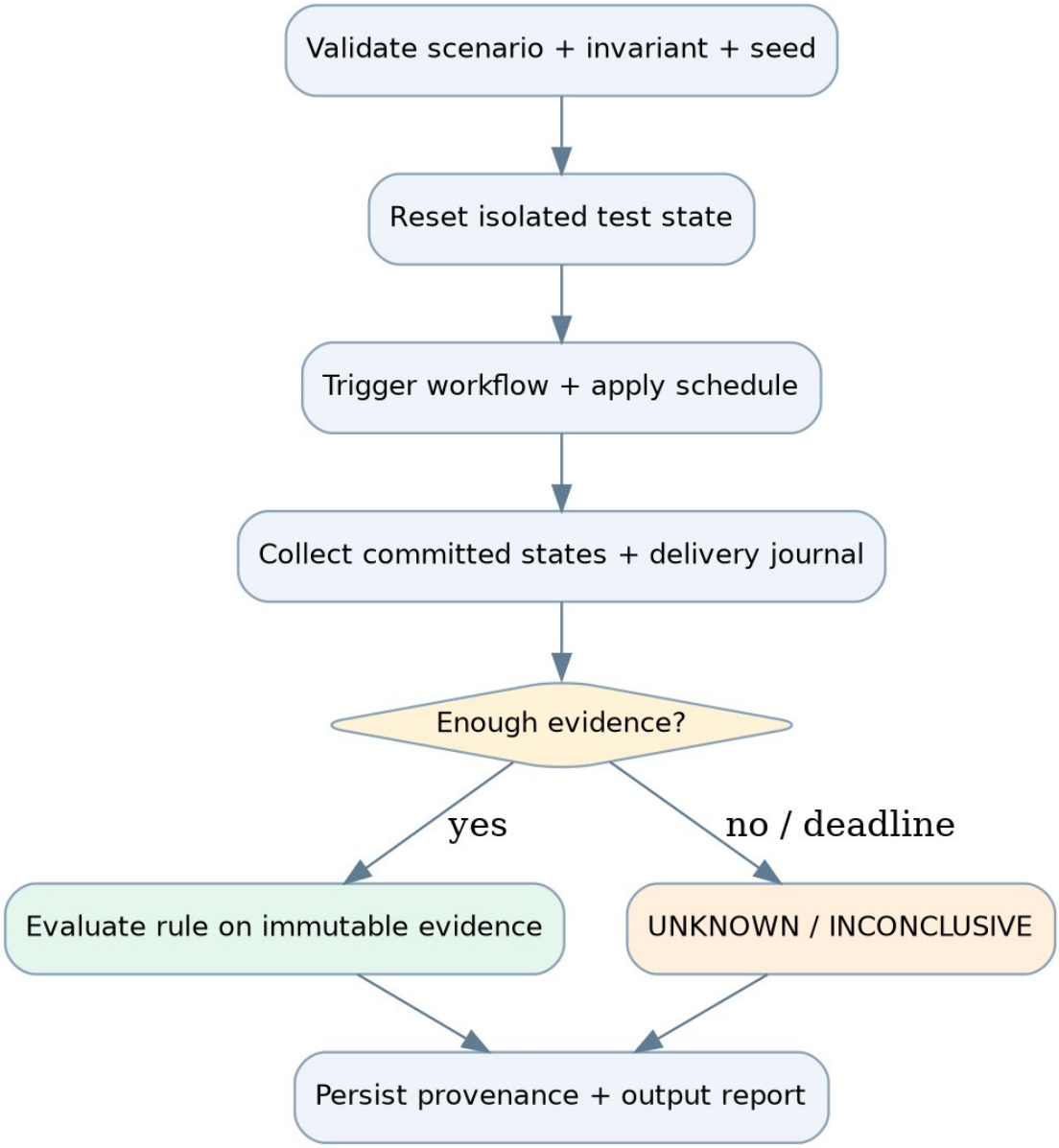


Figure 2. Scenario execution and evidence-based decision flow.

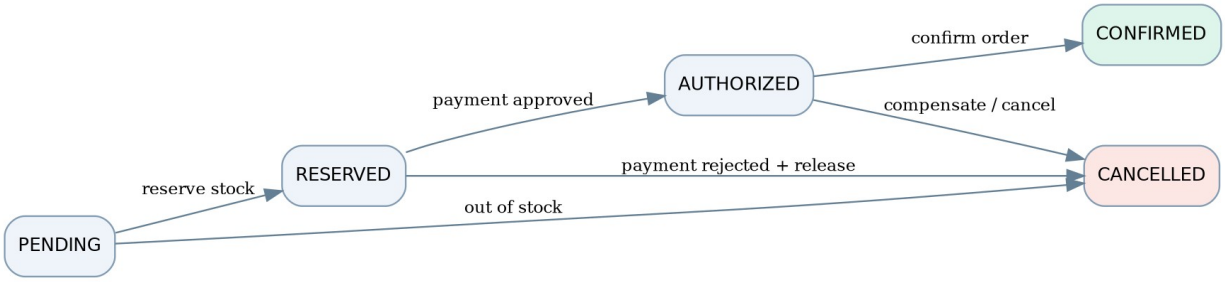


Figure 3. Simplified checkout order state transitions (payment and inventory have separate authoritative states).

Dispatcher interface (conceptual)

schedule(runId, eventKey, intervention), onConsumed(record), hold(token), release(token), duplicate(token, copies), delay(token, duration), journal(runId), drainAndReset(runId). Every release is associated with a delivery attempt ID and records handler acknowledgment/failure independently of the business event ID.

Concurrency model. Per-run controlled queues and logical checkpoints prevent interventions from leaking between runs. Multiple Kafka partitions or competing consumers invalidate global ordering guarantees; MVP limits each controlled scenario to a declared partition/consumer topology, and reports unsupported interleavings as UNKNOWN or configuration error.

Success check for duplicate delivery: identical business event identifier reaches payment processing twice while payment capture count remains one on the corrected implementation. The deliberately broken version produces two captures; no real money is moved.

5. Invariant semantics and verdict model

Separate infrastructure execution status (COMPLETED / TIMED_OUT / INFRA_ERROR) from rule evaluation (SATISFIED / VIOLATED / UNKNOWN). Final CLI verdict is derived by an explicit precedence rule; recorded confirmed violations must not be erased by later report-generation errors.

Rule type	Illustrative assertion	Evidence needed
State invariant	If order=CONFIRMED, payment must be AUTHORIZED and inventory RESERVED.	Correlated authoritative state observations with declared consistency limits.
Uniqueness	At most one successful capture per business order.	Committed captures grouped by stable order ID.
Bounded eventual	Following rejection, reservation becomes RELEASED within 30s.	Durable rejection reference and time-stamped release evidence.
Negative control	Duplicate delivery with correct idempotency does not violate capture uniqueness.	Actual capture count, not event delivery count.

UNKNOWN is required for missing authoritative data, insufficient observation coverage, or indeterminate event ordering. Temporal rules declare a starting event and time basis; a polling-only observation cannot prove the absence of transient forbidden states.

Example rule specification

Rule ID: SV-002 | Trigger: payment.requested | Invariant: count(committed payment captures for order_id) <= 1 | Deadline: after settlement observation window | Severity: critical | Evidence adapter: payment-state-readonly | Version: 1.

6. Automatic exploration, replay and shrinking

Exploration budget is bounded by maximum schedule length, permitted intervention types, seed, number of runs, and per-run deadline. Generator produces only legal interventions according to the target topology and causal prerequisites. Seed and scheduler version are saved alongside all outcomes.

1. Generate fault schedule candidates (duplicate delivery, delay, controlled restart; add reordering only after establishing feasible prerequisites).

2. Reset isolated business fixtures and scheduler journal before each run.
3. Apply fault schedule through controlled dispatcher and collect committed business evidence.
4. Evaluate versioned invariants; preserve FAIL / UNKNOWN / ERROR separately.
5. On a confirmed violation, attempt reduction using dependency-aware delta debugging.
6. Reexecute each reduced candidate across a configured trial count; measure observed reproduction frequency.
7. Save locally minimized candidate, seed, exact build IDs, evidence hashes, reduction history and regression YAML.

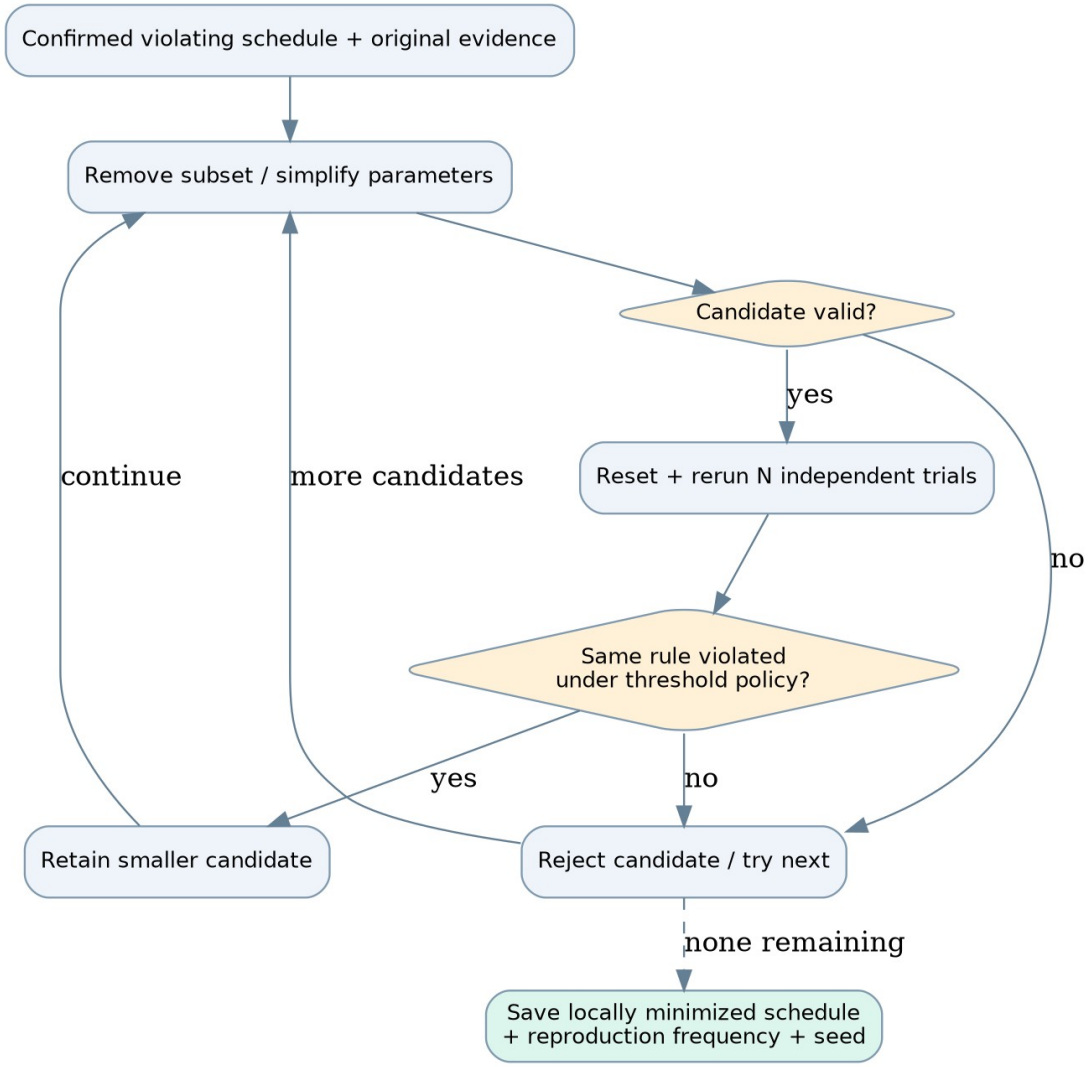


Figure 4. Dependency-aware delta debugging. “Locally minimized” does not mean globally minimal or deterministically repeatable.

Two different replay guarantees

Mode	Input	Guarantee
Evidence replay	Frozen immutable evidence + same rule/compiler version	Deterministic evaluation when parser/evaluator are pure.
Schedule replay	Fault interventions + seed + service builds + reset state	Best-effort application reexecution; quantify reproduction with repeated runs.

Reduction acceptance policy. By default require same invariant violation in at least 4 of 5 trials (a prototype threshold, configurable and not a proof guarantee). Record all pass/unknown/error outcomes to avoid selection bias. Never shrink away causal prerequisites solely to make the schedule shorter.

7. Persistence, data provenance and operational safety

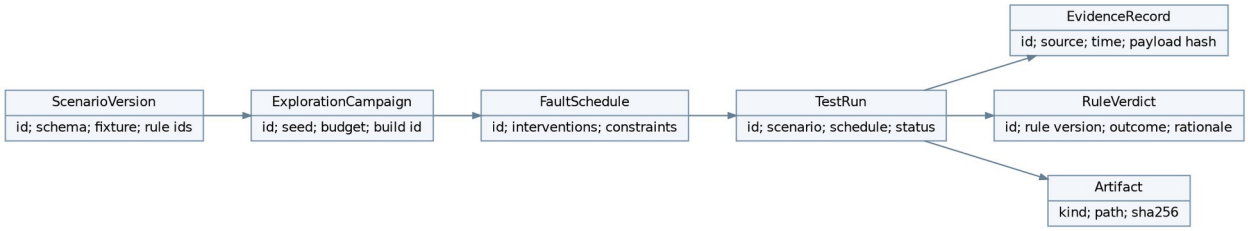


Figure 5. Core logical data model. Store versioned scenario and invariant definitions with each execution.

Suggested entities: ScenarioVersion, InvariantVersion, ExplorationCampaign, FaultSchedule, TestRun, DeliveryAttempt, EvidenceRecord, RuleVerdict, Artifact, ReductionAttempt. Each run stores source commit/build digest, environment fixture version, runner version, schedule seed, clock basis, completion status, and final verdict.

Immutable evidence should include correlation ID, originating service, committed-state source, observation timestamp, payload hash and safe redacted snapshot. Distinguish event time, processing time and observation time. Use UTC in storage and explicit timezone in display.

Isolation. Dedicated compose network, disposable Postgres schemas/databases per run or serialized reset for spike, fixture teardown, timeout boundaries, resource caps, and no production credentials. Normal source-code dependencies must not invoke test scheduler in deployed production mode.

8. API, CLI and repository layout

CLI design

stateverdict run scenarios/checkout.yaml --environment local --report artifacts/run.json

stateverdict explore scenarios/checkout.yaml --seed 4281 --iterations 100 --max-faults 3 --shrink

stateverdict replay-evidence artifacts/run.json --rule rules/sv-002.yaml

stateverdict replay-schedule scenarios/found/sv-002.yaml --trials 5

Exit codes: 0 verified pass; 1 confirmed critical violation; 2 inconclusive; 3 tool/config/infrastructure error. CI default fail-closed for 1–3 with explicit override policy for noncritical rules.

Repository skeleton

apps/stateverdict-core/{scenario,runner,scheduler,evidence,invariants,explorer,shrinker,reporting}; apps/cli; examples/commerce/{order-service,payment-service,inventory-service}; scenarios/{baseline,found}; infrastructure/compose; tests/{unit,integration,mutation,negative-controls}; docs/{architecture,decisions,evaluations}; .github/workflows; DEVELOPMENT_CHECKPOINT.md

Integrations

First implementation is CLI-driven. GitHub Actions executes deterministic known regression schedules on pull requests; randomized campaigns run in separate scheduled/manual CI jobs because runtime and variability are less suitable as a mandatory fast PR gate.

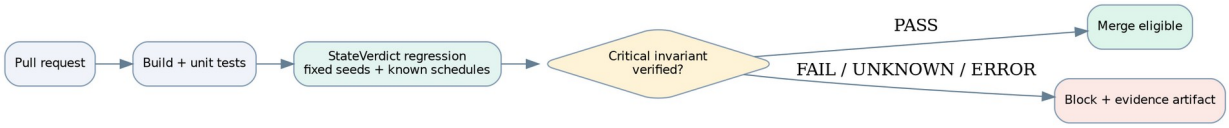


Figure 6. Proposed CI policy for known, stable regression schedules.

9. Test strategy and evaluation plan

Layer	Examples	Release condition
Unit	Pure evaluation, rule parsing, scheduler plan validation, shrinker dependency checks	Deterministic tests pass.
Integration	Kafka record -> scheduler -> handler; PostgreSQL committed capture check	Correct and vulnerable variants behave as specified.
Negative controls	Duplicate delivery on fixed idempotent consumer; harmless delay	No false violation.
Repeated trials	Ten clean resets with fixed schedule; later varied seeds	Report successes, failures, unknown and infrastructure errors.
Mutation evaluation	Idempotency removal, compensation omission, late event, missing acknowledgement	Measure detection and misses with matched baseline.
External validation	One separately authored open-source service with reachable state adapter	Demonstrate generalization; no false novelty claim.

Key metrics: TP/FP/TN/FN where truth is independently defined, UNKNOWN and ERROR frequency, bug detection per fixed time budget, schedule reproduction rate, shrink size change, median run latency, run-to-run variance, and evidence completeness. Results must publish denominators and the exact corpus and commits.

Fair baseline. Compare against JUnit + Testcontainers + Awaitility provided with comparable access to business state and similar compute budget. Do not claim superiority based only on more test executions.

10. Build sequence: seven-day spike then gated roadmap

Spike day	Implementation	Pass/fail demonstration
1	Bootstrap repo, Compose, two services, Postgres, Kafka	Health checks and isolated test fixtures work.
2	Normal checkout flow with committed capture	Exactly one capture for one business request.
3	Scheduler intercept, hold, release and duplicate	Event journal proves two deliveries of same business event.
4	Intentionally vulnerable handler + count invariant	A duplicate delivery can yield two committed captures and FAIL.
5	Ten clean-state repetitions	Record actual reproduction frequency, UNKNOWN and errors.

6	Corrected idempotent handler	Two deliveries yield only one committed capture and PASS.
7	CLI automation, report, docs, repeatable compose setup	Another developer can run A/B/C proof without manual intervention.

Spiking 25–35 hours across seven calendar days is aggressive, especially if learning Kafka. If the acceptance test fails, do not advance to frontend or broader scope; document the blocker and remedy.

Phase	Approx. effort	Exit gate
Core spike and evidence CLI	45–65 h	First trusted FAIL and corrected PASS.
Asynchronous / temporal testing	35–50 h	Bounded eventual assertions on Kafka workflow.
Constrained exploration	35–55 h	Generated schedule finds a violation not hardcoded as scenario.
Shrinker and reproducibility	40–65 h	Locally minimized schedule with repeated-trial frequency.
Evaluation, CI, second system (if feasible)	40–60 h	Controls, comparative report, regression gate.
Documentation / demo	15–25 h	Reproducible README and recruiter demonstration.

Total estimated effort: 210–320 hours, roughly 12–16 weeks at ~20–25 focused hours per week. This is a planning estimate; do not treat it as a guaranteed deadline.

11. Decisions, risks, and explicit limitations

ADR	Decision / reason
ADR-001	Platform core is modular monolith; demo SUT contains services, avoiding unnecessary platform microservices.
ADR-002	Dispatcher is a controlled test-mode scheduler AFTER Kafka consumption, not Kafka internals.
ADR-003	Committed state evidence beats inference from tracing; traces remain supporting diagnostic material.
ADR-004	Execution and rule truth are separate; missing evidence never yields PASS.
ADR-005	Replay of evidence is deterministic; replay of distributed schedules is statistical.
ADR-006	Use dependency-aware ddmin for shrinker; build only after reliable execution.
ADR-007	CLI, reproducibility and CI before dashboard/cloud.

Risk	Mitigation
False positives from incomplete state	UNKNOWN with stated coverage; negative controls

	and independent evidence.
Scheduler leaks into business code	Test-only adapter boundary; production bypass and contract tests.
Race conditions prevent replay	Controlled checkpoints + repeated trials; report frequency.
State leakage between runs	Per-run IDs, reset hooks, database isolation, ordered cleanup.
Random exploration budget explodes	Constrained generator, bounded iterations, phase-gated features.
Circular self-testing	Independent mutation corpus, controls, external target when viable.
Premature UI overbuild	No UI until measured signature feature passes gates.

12. Developer handoff, checkpoint policy and resume demo

Session exit checklist

- Commit and push working state; capture branch, commit SHA and clean/dirty tree status.
- Record last green command and the exact failing command with relevant error excerpt.
- Mark latest completed acceptance criterion with evidence link or report path.
- Record active blocker, suspected cause, and one narrowly scoped next action.
- Add an ADR if architecture/contract changed; update this specification version when scope changes.

Definition of Done for any feature

Implementation compiles; focused unit tests pass; integration behavior verified against real dependencies where relevant; negative case covered; docs/examples updated; no fake metrics; reproducible command captured in checkpoint.

Recruiter demo in 60 seconds

1. Explain: “An order can be confirmed despite failed payment even if each service appears healthy.” 2. Show the same event delivered twice. 3. Show vulnerable implementation with two committed captures and StateVerdict FAIL. 4. Enable idempotency and show one capture and PASS. 5. Show captured schedule and evidence. Later: show automated discovery and local minimization with actual measured results.

Resume wording policy

Use “built a prototype” or “engineered a developer testing tool” until there is real adoption. No fabricated defect-detection rate, customer count, or novel invention claim. Signature claim only after generated fault exploration is demonstrably operational.

Appendix A — Minimal failure report contract

Fields: run_id, scenario_version, rule_version, source_commit, schedule_seed, fault_schedule, execution_status, evaluation_status, verdict, violation_id, evidence_refs, observations, reproduction_trials, reproduction_frequency, artifact_checksums, started_at, completed_at.

Appendix B — Open questions to resolve during spike

- Kafka partition/consumer topology permitted for strict controlled dispatch in v1.
- Whether to use dedicated service DBs or schemas for local spike; ensure actual service ownership.

- Exact source of committed payment captures and reset transaction boundary.
- Clock model for bounded temporal assertions; monotonic run timers vs event timestamps.
- Definition of a valid and shrinkable schedule dependency graph.
- Which existing open-source target, if any, allows ethical isolated fault testing without sensitive data.